Computer Programming Using C(PC)
DU # 2301CS101

Darshan UNIVERSITY
योग: कर्मसु कौशलम्

Unit-4
Functions





Prof. Vishal Kansagara
Computer Engineering Department
Darshan Institute of Engineering & Technology, Rajkot
Vishal.kansagara@darshan.ac.in
8200601076

Outline

- Function
- Types of function
- Actual parameters and formal parameters
- Return statement
- Category of function
- Advantages of function
- Recursion

Function

Darshan UNIVERSITY

What is Function?

- A function is a group of statements that perform a specific task.
- It divides a large program into smaller parts.
- A function is something like hiring a person to do a specific job for you.
- Every C program can be thought of as a collection of these functions.
- Program execution in C language starts from the main function.

Syntax

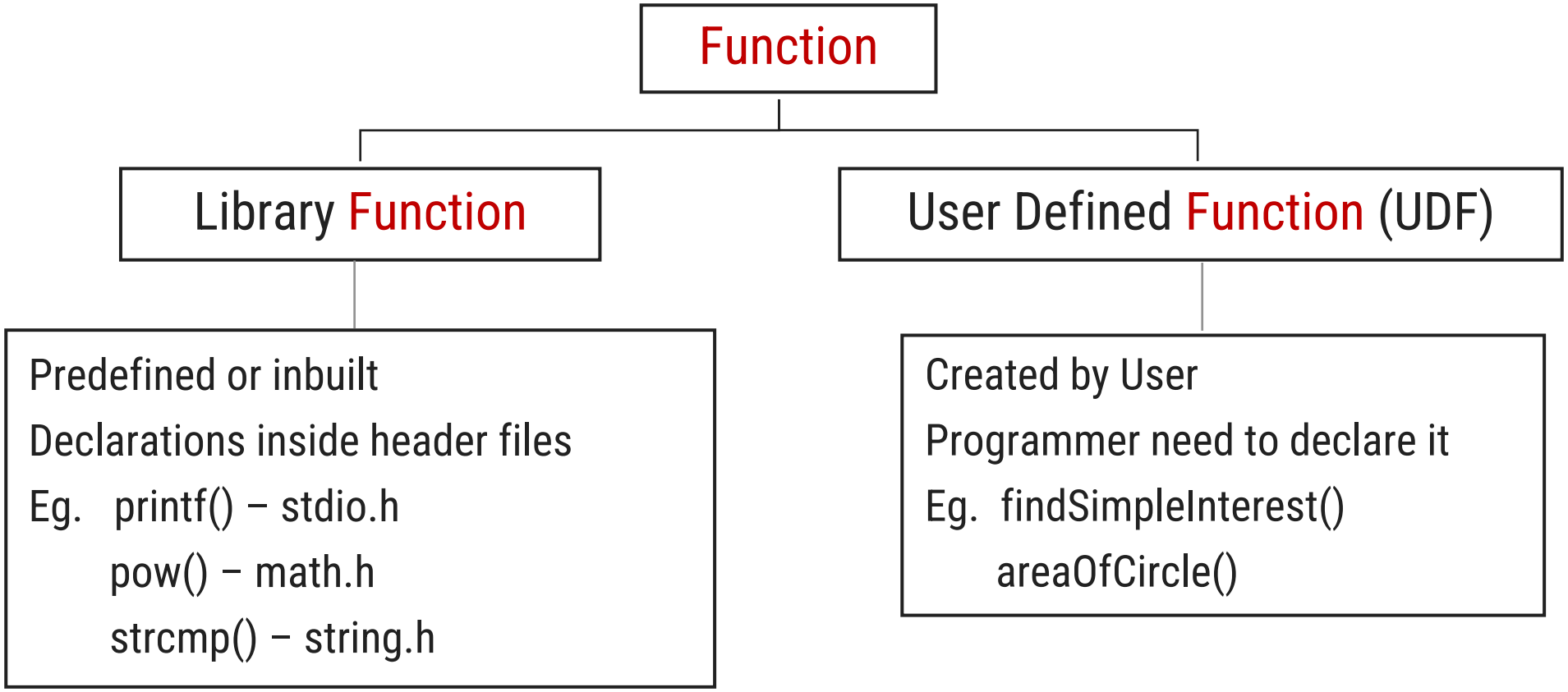
```
void main()
{
   // body part
}
```

- Why function ?
  - Avoids rewriting the same code over and over.
  - Using functions it becomes easier to write programs and keep track of what they doing.

Types of function

Darshan UNIVERSITY

Types of Function



Program Structure for Function

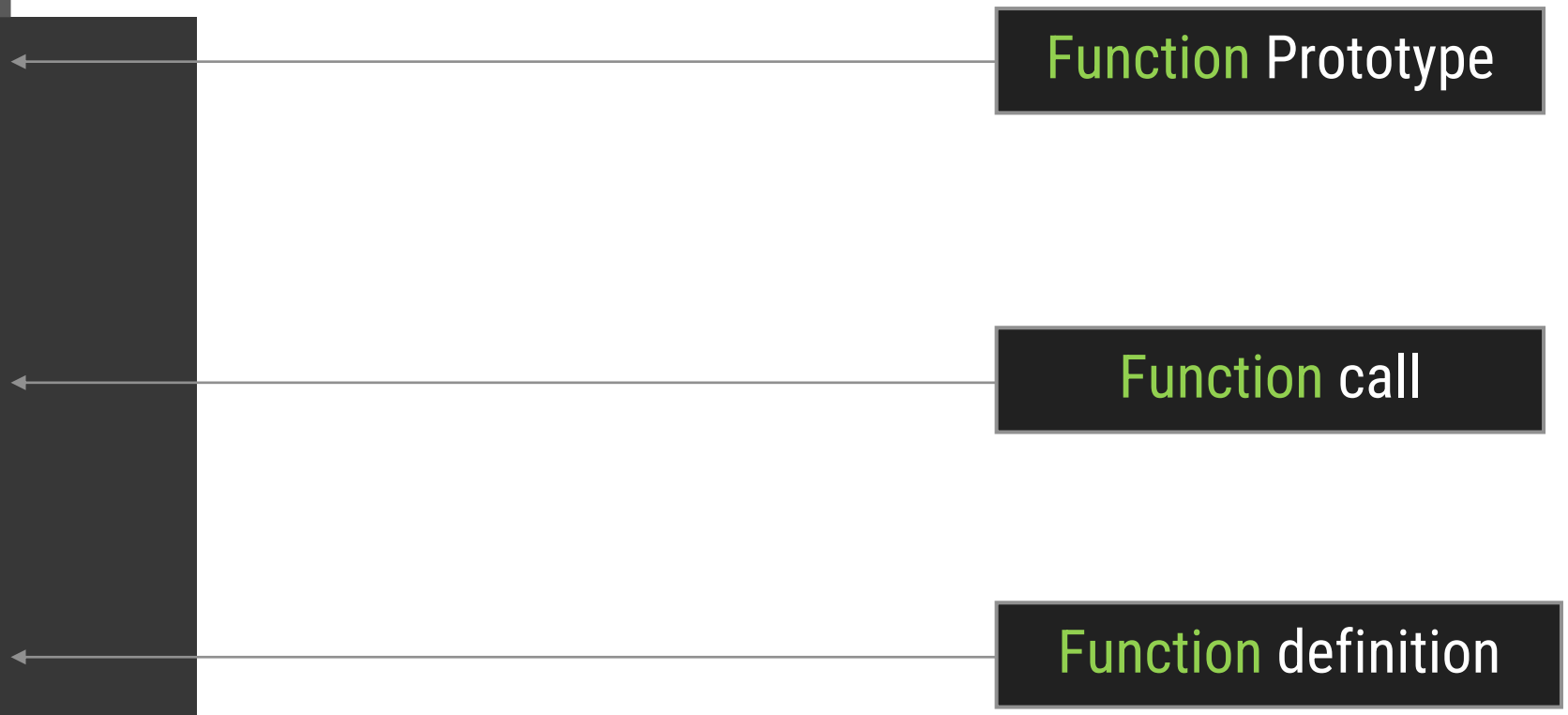
- When we use a user-defined function program structure is divided into three parts.

Function Structure

```
void func1();


void main()
{
  ....
  func1();
}

void func1()
{
  ....
  //function body
  ....
}
```



Function Prototype

- A function Prototype also know as function declaration.
- A function declaration tells the compiler about a function name and how to call the function.
- It defines the function before it is being used or called.
- A function prototype needs to be written at the beginning of the program.

Syntax

```
return-type function-name (arg-1, arg 2, …);
```

Example

```
void addition(int, int);
```

Darshan UNIVERSITY

Function Definition

- A function definition defines the functions header and body.
- A function header part should be identical to the function prototype.
  - Function return type
  - Function name
  - List of parameters
- A function body part defines function logic.
  - Function statements

Syntax

```
return-type function-name (arg-1, arg 2, …)
{
    //… Function body
}
```

Example

```
void addition(int x, int y)
{
 printf("Addition is=%d“,(x+y)); }
```

Darshan UNIVERSITY

WAP to add two number using add(int, int) Function

Program

```c
#include <stdio.h>
void add(int, int); // function declaration
void main()
{
int a = 5, b = 6;
add(a, b); // function call
}

void add(int x, int y) // function definition
{
   printf("Addition is = %d", x + y);
}
```

Output

```
Addition is = 11
```

Actual & formal arguments

Darshan UNIVERSITY

Actual parameters and Formal parameters

- Values that are passed to the called function from the main function are known as Actual parameters.
- The variables declared in the function prototype or definition are known as Formal parameters.
- When a method is called, the formal parameter is temporarily "bound" to the actual parameter.

Return statement

Darshan UNIVERSITY

Return Statement

- If function is returning a value to calling function, it needs to use the keyword return.
- The called function can only return one value per call.

Syntax

```
return;
                Or

return (expression);
```

WAP to find maximum number from two number

Program

```c
#include <stdio.h>
int max(int a, int b);
void main(){
    int a = 100;
    int b = 200;
    int maxvalue;
    maxvalue = max(a, b);
    printf("Max value is : %d\n", maxvalue);
}
int max(int a, int b){
    if (a > b)
        return a; // return a
    else
        return b; // return b
}
```

Output

```
Max value is : 200
```

WAP to calculate the Power of a Number

Program

```c
#include <stdio.h>
int power(int, int);
void main()
{
    int num, pow, res;
    printf("Enter any number : ");
    scanf("%d", &num);
    printf("Enter power of number : ");
    scanf("%d", &pow);
    res = power(num, pow);
    printf("%d's power %d = %d", num, pow, res);
}
int power(int n, int p){
    int r = 1;
    while (p >= 1){
        r = r * n;
        p--;
    }
    return r;
}
```

Output

```
Enter any number : 5
Enter power of number : 3
5's power 3 = 125
```

Darshan UNIVERSITY

WAP to find Factorial of a Number

Program

```c
#include <stdio.h>
int fact(int);
int main(){
        int n, f;
        printf("Enter the number :\n");
        scanf("%d", &n);
        f = fact(n);
        printf("factorial = %d", f);
}
int fact(int n){
        int i, fact = 1;
        for (i = 1; i <= n; i++)
                fact = fact * i;
        return fact;
}
```

Output

```
Enter the number :
5
factorial = 120
```

WAP to check Number is Prime or not

Program

```c
#include <stdio.h>
int checkPrime(int);
void main()
{
    int n1, prime;
    printf("Enter the number :");
    scanf("%d", &n1);
    prime = checkPrime(n1);
    if (prime == 1)
        printf("The number %d is a prime number.\n", n1);
    else
        printf("The number %d is not a prime number.\n", n1);
}
```

Program contd.

```c
int checkPrime(int n1)
{
        int i = 2;
        while (i <= n1 / 2)
        {
                if (n1 % i == 0)
                        return 0;
                else
                        i++;
        }
        return 1;
}
```

Output

```
Enter the number :7
The number 7 is a prime number.
```

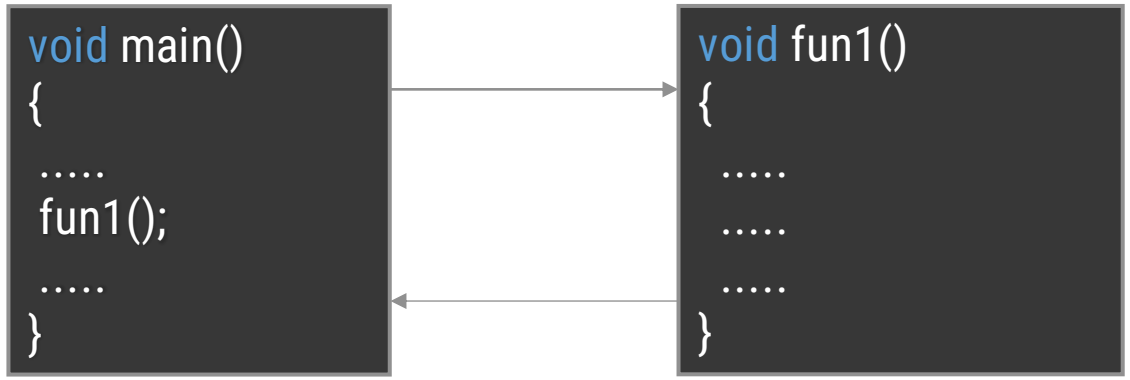
Darshan UNIVERSITY

Categories of function

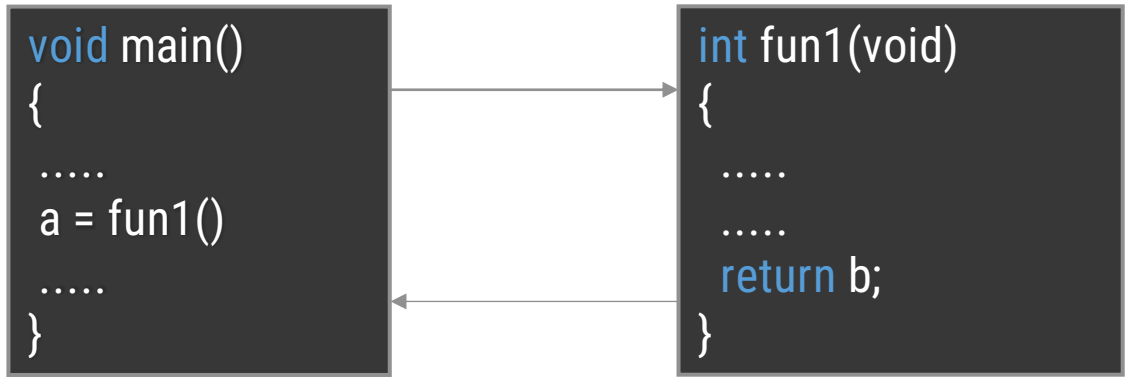
Darshan UNIVERSITY

Category of Function

(1) Function with no argument and but no return value

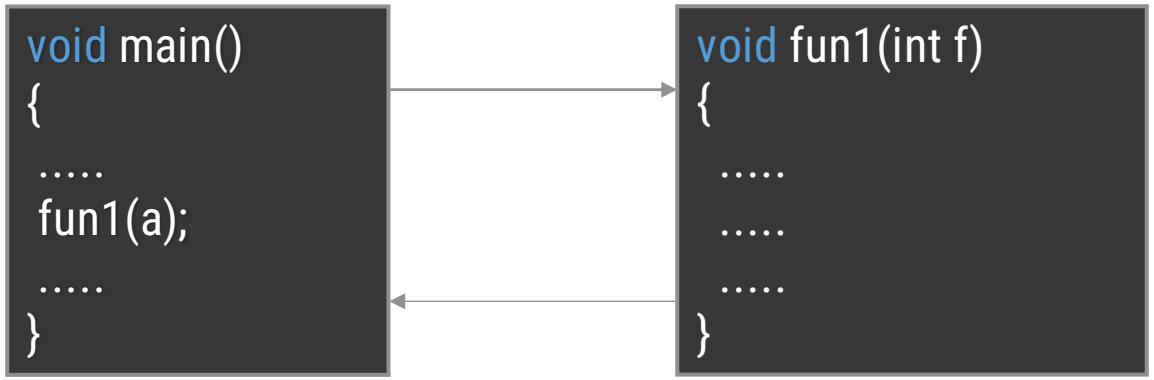


(2) Function with no argument and returns value

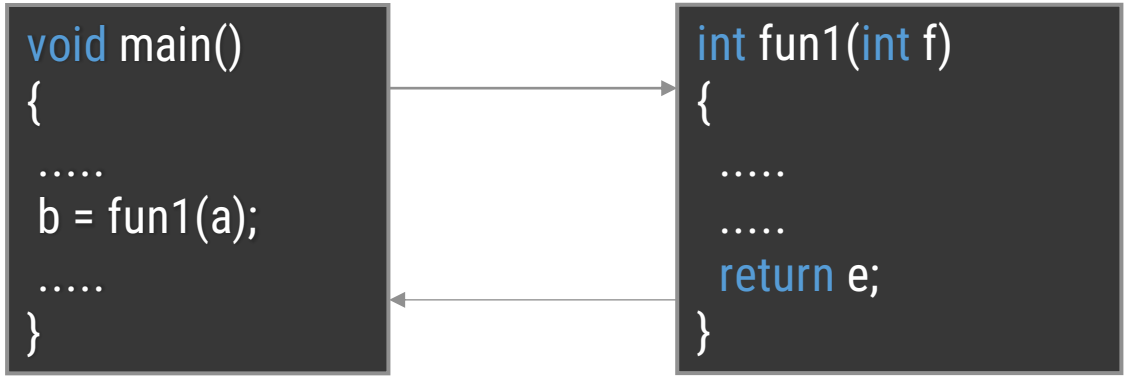


Category of Function cont.

(3) Function with argument and but no return value



(4) Function with argument and returns value



Advantages of function

Darshan UNIVERSITY

Advantages of Function

- Using function we can avoid rewriting the same logic or code again and again in a program.
- We can track or understand large program easily when it is divide into functions.
- It provides reusability.
- It help in testing and debugging because it can be tested for errors individually in the easiest way.
- Reduction in size of program due to code of a function can be used again and again, by calling it.

Darshan UNIVERSITY

Variables

- Variable is a symbolic name given to some value which can be changed.
- $x, y, a, count, etc.$ can be variable names.
- x=5 a=b+c
- There are many types of variables in c:
  - Local variable:
    - A variable which is declared inside the function is known as local variable. It is used only inside the function in which it is declared.
  - Global variable:
    - A variable which is declared outside the function is known as global variable. It can be used throughout the program.
  - Static variable:
    - It is used to retain its value between multiple function calls. It is declared using static keyword.
  - External variable:
    - You can share a variable in multiple C source files by using external variable. It is declared using extern keyword.
  - Automatic variable:
    - Variable which is declared inside the block is known as automatic variable by default.

Storage Classes

- Storage class decides the scope, lifetime and memory allocation of variable.
- Scope of a variable is the boundary within which a variable can be used.

Storage Specifier	Storage	Initial Value	Scope	Life	Example
Automatic {auto}	Stack	Garbage	Within block	End of block	int a; auto int a;
Register {register}	CPU register	Garbage	Within block	End of block	register int var;
External {extern}	Data segment	Zero	Global Multiple file	Till end of program	extern int var;
Static {static}	Data segment	Zero	Within block	Till end of program	static extern int var; static int var;

Pointer and Function

- Like normal variable, pointer variable can be passed as function argument and function can return pointer as well.
- There are two approaches to passing argument to a function:
  - Call by value
  - Call by reference / address

Call by Value

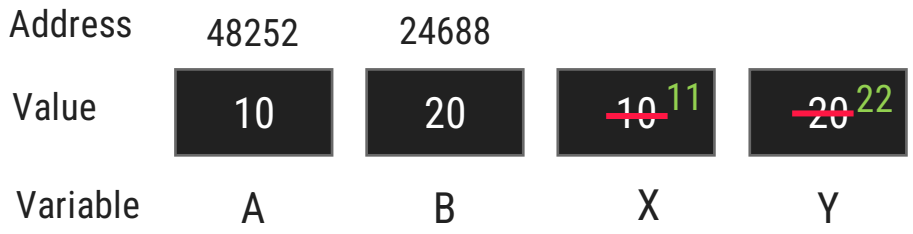
- In this approach, the values are passed as function argument to the definition of function.

Program

```
1  #include<stdio.h>
2  void fun(int,int);
3  int main()
4  {
5     int A=10,B=20;
6     printf("\nValues before calling %d, %d",A,B);
7     fun(A,B);
8     printf("\nValues after calling %d, %d",A,B);
9     return 0;
10 }
11 void fun(int X,int Y)
12 {
13    X=11;
14    Y=22;
15 }
```

Output

```
Values before calling 10, 20
Values after calling 10, 20
```



Call by Reference / Address

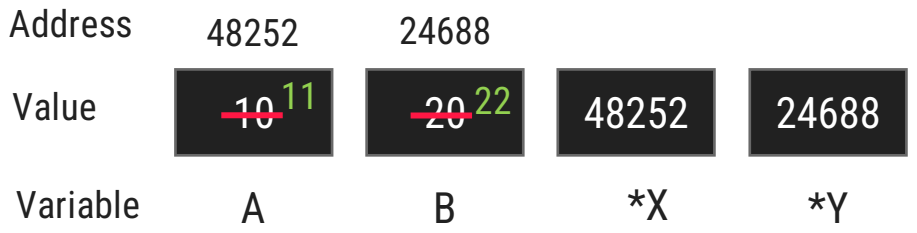
- In this approach, the references / addresses are passed as function argument to the definition of function.

Program

```
1  #include<stdio.h>
2  void fun(int*,int*);
3  int main()
4  {
5     int A=10,B=20;
6     printf("\nValues before calling %d, %d",A,B);
7     fun(&A,&B);
8     printf("\nValues after calling %d, %d",A,B);
9     return 0;
10 }
11 void fun(int *X,int *Y)
12 {
13    *X=11;
14    *Y=22;
15 }
```

Output

```
Values before calling 10, 20
Values after  calling 11, 22
```



WAP to swap value of two variables using pointer / call by reference.

Program

```c
int main(){
    int num1,num2;
    printf("Enter value of num1 and num2: ");
    scanf("%d %d",&num1, &num2);

    //displaying numbers before swapping
    printf("Before Swapping: num1 is: %d, num2 is:
%d\n",num1,num2);

    //calling the user defined function swap()
    swap(&num1,&num2);

    //displaying numbers after swapping
    printf("After Swapping: num1 is: %d, num2 is:
%d\n",num1,num2);
    return 0;
}
```

```c
void swap(int *num1,int *num2)
{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
```

Output

```
Enter value of num1 and num2: 5
10
Before Swapping: num1 is: 5, num2 is: 10
After  Swapping: num1 is: 10, num2 is: 5
```

Darshan UNIVERSITY

Pointer to Function

- Every function has reference or address, and if we know the reference or address of function, we can access the function using its reference or address.
- This is the way of accessing function using pointer.

Syntax

```
return-type (*ptr-function)(argument list);
```

- return-type: Type of value function will return.
- argument list: Represents the type and number of value function will take, values are sent by the calling statement.
- (*ptr-function): The parentheses around *ptr-function tells the compiler that it is pointer to function.
- If we write *ptr-function without parentheses then it tells the compiler that ptr-function is a function that will return a pointer.

Write a program to sum of two numbers using pointer to function.

Program

```c
#include<stdio.h>
int Sum(int,int);
int (*ptr)(int,int);
int main()
{
   int a,b,rt;
   printf("\nEnter 1st number : ");
   scanf("%d",&a);
   printf("\nEnter 2nd number : ");
   scanf("%d",&b);
   ptr = Sum;
   rt = (*ptr)(a,b);
   printf("\nThe sum is : %d",rt);
   return 0;
}
int Sum(int x,int y)
{
   return x + y;
}
```

Output

```
Enter 1st number : 5

Enter 2nd number : 10

The sum is : 15
```

Darshan UNIVERSITY

Practice Programs

1) WAP to count simple interest using function.
2) WAP that defines a function to add first *n* numbers.
3) WAP using global variable, static variable.
4) WAP that will scan a character string passed as an argument and convert all lowercase character into their uppercase equivalents.
5) Build a function to check number is prime or not. If number is prime then function return value 1 otherwise return 0.
6) Write a program to calculate nCr using user defined function. nCr = n! / (r! * (n-r)!)
7) Create a function to swap the values of two variables.
8) Write a function which takes 2 numbers as parameters and returns the gcd of the 2 numbers. Call the function in main().

Recursion

Darshan UNIVERSITY

What is Recursion?

- Any function which calls itself is called recursive function and such function calls are called recursive calls.
- Recursion cannot be applied to all problems, but it is more useful for the tasks that can be defined in terms of a similar subtask.
- It is idea of representing problem a with smaller problems.
- Any problem that can be solved recursively can be solved iteratively.
- When recursive function call itself, the memory for called function allocated and different copy of the local variable is created for each function call.
- Some of the problem best suitable for recursion are
  - Factorial
  - Fibonacci
  - Tower of Hanoi

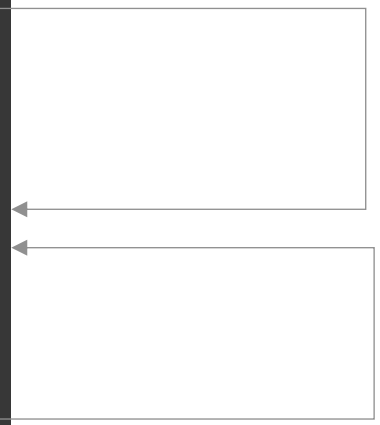
Working of Recursive function

Working

```
void func1();


void main()
{
   ....
   func1();
   ....
}

void func1()
{
   ....
   func1();
   ....
}
```



Function call

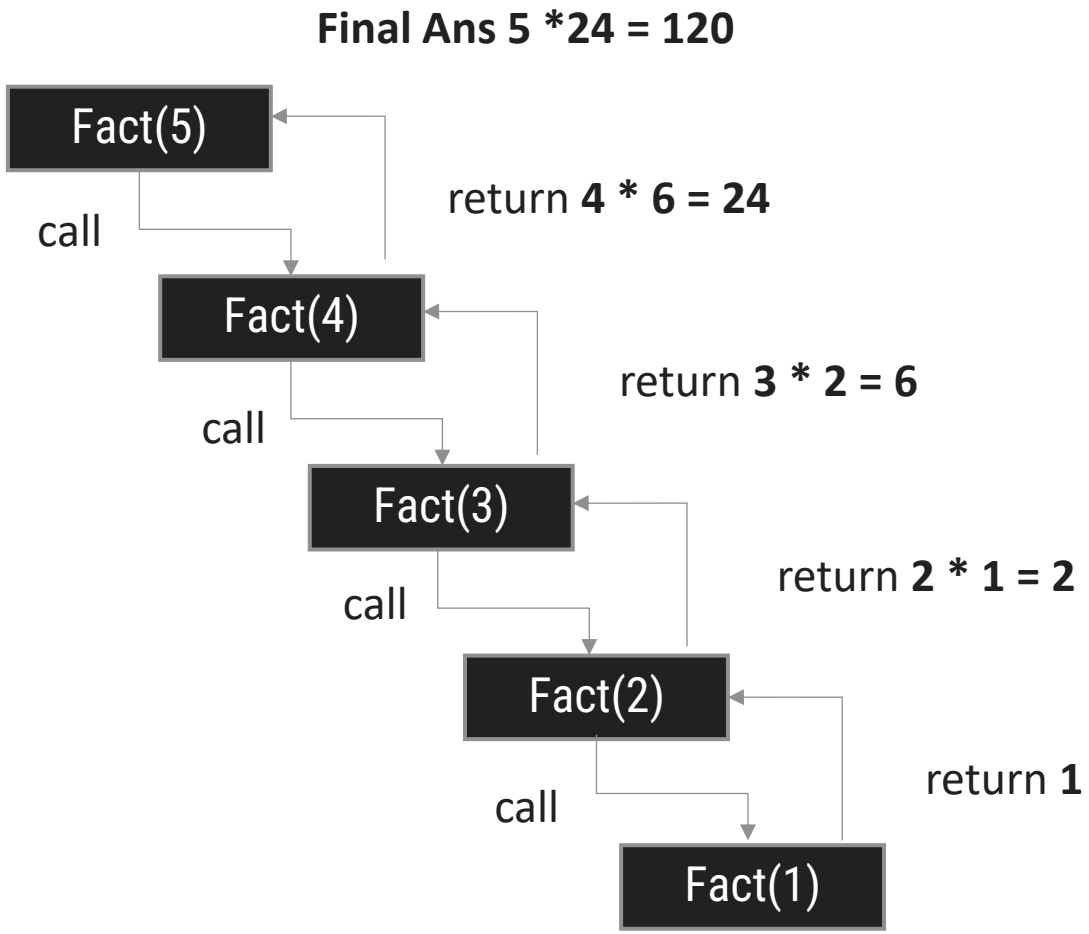
Recursive function call

Properties of Recursion

- A recursive function can go infinite like a loop. To avoid infinite running of recursive function, there are two properties that a recursive function must have.
- Base Case or Base criteria
  - It allows the recursion algorithm to stop.
  - A base case is typically a problem that is small enough to solve directly.
- Progressive approach
  - A recursive algorithm must change its state in such a way that it moves forward to the base case.

Recursion - factorial example

- The factorial of a integer n, is product of
  - n * (n-1) * (n-2) * …. * 1
- Recursive definition of factorial
  - n! = n * (n-1)!
  - Example
    - 3! = 3 * 2 * 1
    - 3! = 3 * (2 * 1)
    - 3! = 3 * (2!)



WAP to find factorial of given number using Recursion

Program

```c
#include <stdio.h>
int fact(int);
void main(){
    int n, f;
    printf("Enter the number:\n");
    scanf("%d", &n);
    f = fact(n);
    printf("factorial = %d", f);
}
int fact(int n){
    if (n == 0)
        return 1;
    else if (n == 1)
        return 1;
    else
        return n * fact(n - 1);
}
```

Output

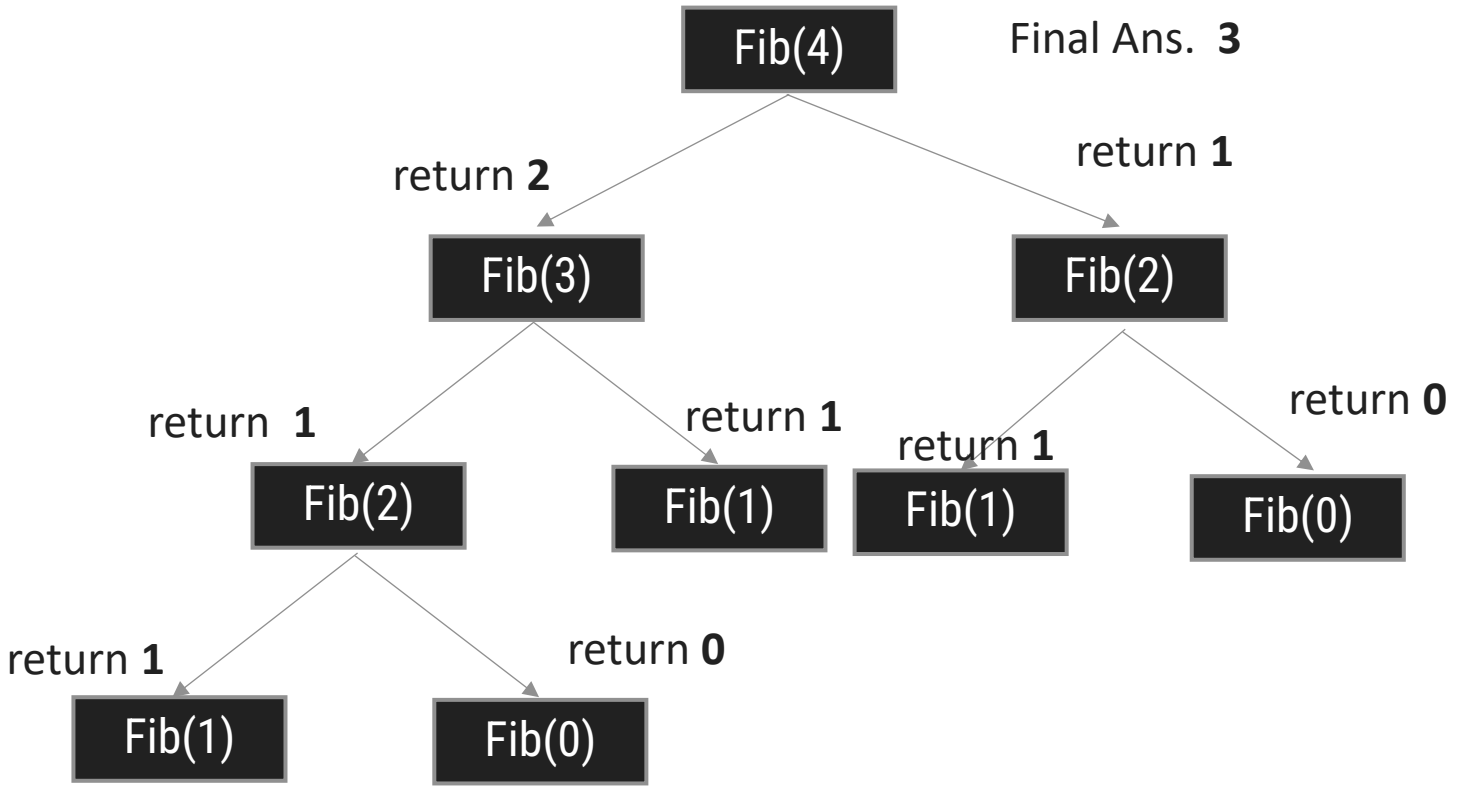
```
Enter the number: 5
factorial = 120
```

Darshan UNIVERSITY

Recursion - Fibonacci example

- A series of numbers , where next number is found by adding the two number before it.
- Recursive definition of Fibonacci
  - Fib(0) = 0
  - Fib(1) = 1
  - Fib(n) = Fib(n-1) + Fib(n-2)
- Example
  - Fib(4) = Fib(3) + Fib(2)
  - Fib(4) = 3

Recursive trace



WAP to Display Fibonacci Sequence

Program

```c
#include <stdio.h>
int fibonacci(int);
void main(){
    int n, m = 0, i;
    printf("Enter Total terms\n");
    scanf("%d", &n);
    printf("Fibonacci series\n");
    for (i = 1; i <= n; i++){
        printf("%d ", fibonacci(m));
        m++;
}
}
```

Program contd.

```c
int fibonacci(int n){
    if (n == 0 || n == 1)
        return n;
    else
        return (fibonacci(n - 1) +
    fibonacci(n - 2));
}
```

Output

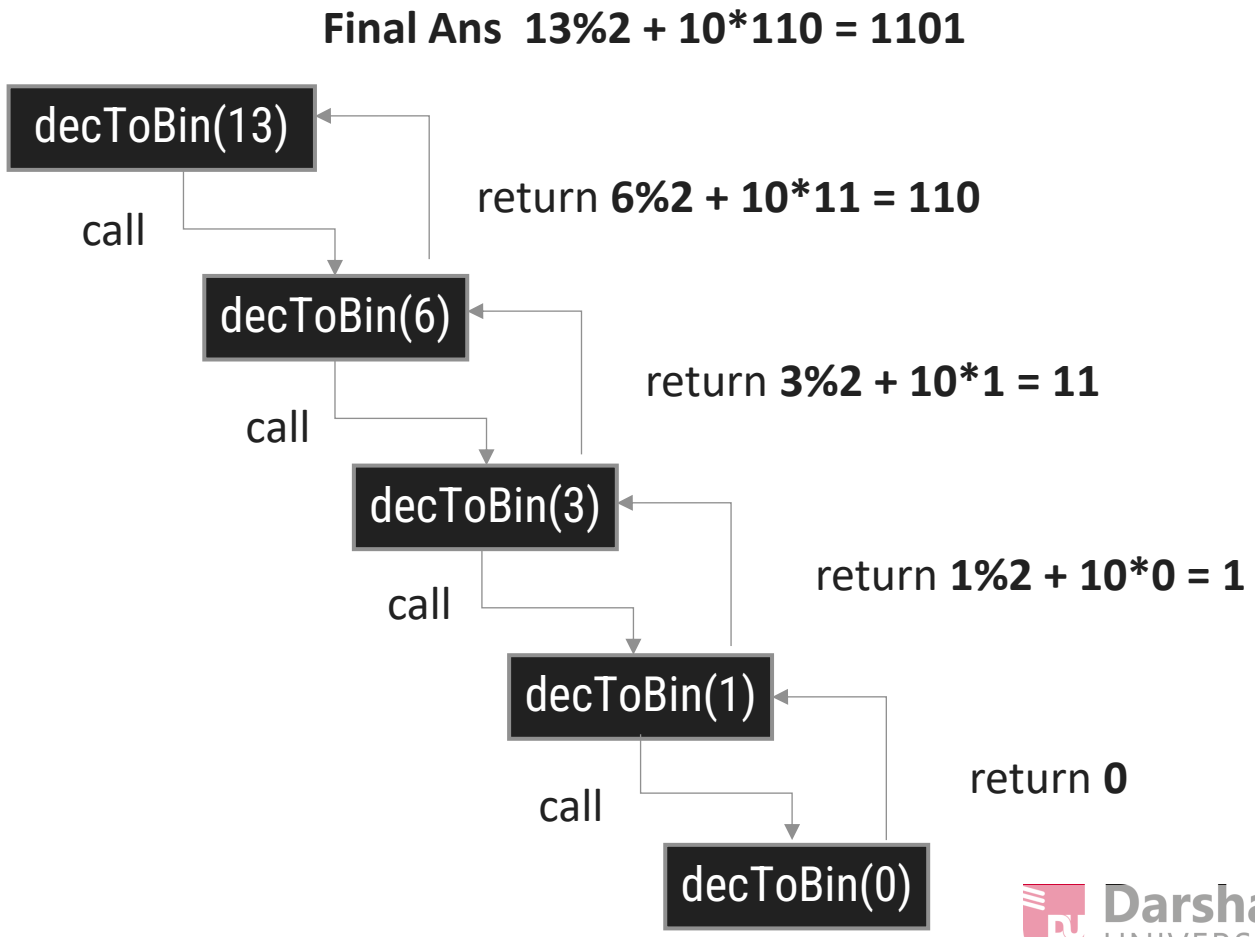
```
Enter Total terms
5
Fibonacci series
0 1 1 2 3
```

Darshan UNIVERSITY

Recursion - Decimal to Binary example

- To convert decimal to binary, divide decimal number by 2 till dividend will be less then 2
- To convert decimal 13 to binary
  - 13/2 = 6 reminder 1
  - 6/2 = 6 reminder 0
  - 3/2 = 3 reminder 1
  - 1/2 = 1 reminder 1
- Recursive definition of Decimal to Binary
  - decToBin(0) = 0
  - decToBin(n) = n%2 + 10* decToBin(n/2)
- Example
  - decToBin(13) = 13%2 + 10 decToBin(6)
  - decToBin(13) = 1101

Recursive trace



WAP to Convert Decimal to Binary

Program

```c
#include <stdio.h>
int convertDecimalToBinary(int);
void main(){
    int dec, bin;
    printf("Enter a decimal number: ");
    scanf("%d", &dec);
    bin = convertDecimalToBinary(dec);
    printf("The binary equivalent = %d \n",bin);
}
int convertDecimalToBinary(int dec){
    if (dec == 0)
        return 0;
    else
        return (dec % 2 + 10 *
convertDecimalToBinary(dec / 2));
}
```

Output

```
Enter a decimal number: 12
The binary equivalent = 1100
```

Darshan UNIVERSITY

WAP to Convert Binary to Decimal

Program

```c
#include <stdio.h>
int convertBinaryToDecimal(int b, int c, int t);
void main(){
    unsigned int binary, decimal;
    printf("Enter a binary number: ");
    scanf("%d", &binary);
    decimal = convertBinaryToDecimal(binary, 1, 0);
    printf("Decimal value of %d is %d", binary, decimal);
}
int convertBinaryToDecimal(int b, int c, int t){
    if (b > 0){
        t += (b % 10) * c;
        convertBinaryToDecimal(b / 10, c * 2, t);
    }
    else
        return t;
}
```

Output

```
Enter a binary number: 101
Decimal value of 101 is 5
```

Darshan UNIVERSITY

Practice Programs

1) Write a program to find factorial of a given number using recursion.
2) WAP to convert decimal number into binary using recursion.
3) WAP to use recursive calls to evaluate $F(x) = x - x^3/3! + x^5/5! - x^7/7! + \ldots + x^n/n!$

Thank you

Darshan UNIVERSITY